VidyaTrack Frontend Feature-by-Feature Walkthrough

1. How To Use This Walkthrough

This document explains the frontend feature by feature.

For each major area, it answers:

- what the feature does
- which files matter
- how data flows
- what the user sees
- what to modify if requirements change

2. Auth Surface

Purpose

Handles OTP login and school selection.

Main files

- `src/features/auth/OtpRequestPage.tsx`
- `src/features/auth/OtpVerifyPage.tsx`
- `src/features/auth/components/AuthTopBar.tsx`
- `src/features/auth/otpRequest/hooks/useOtpRequestPage.ts`
- `src/features/auth/otpVerify/hooks/useOtpVerifyPage.ts`

Flow

1. user enters phone number
2. OTP request mutation runs
3. app navigates to verify page
4. user enters OTP
5. token is stored
6. `/me` is fetched
7. user is redirected by role
8. if multiple schools apply, user can select school

Important notes

- auth errors are preserved exactly
- OTP digit behavior is managed in hooks
- school selection is part of the auth entry path

3. Platform Surface

3.1 Platform Dashboard

Purpose:

- high-level super-admin overview

3.2 Schools List

Main files:

- `src/features/platform/PlatformSchoolsListPage.tsx`
- `src/features/platform/schoolsList/hooks/usePlatformSchoolsListPage.ts`
- `src/features/platform/schoolsList/components/*`

Flow:

1. schools list loads
2. fallback dashboard queries run per school for metrics
3. search filters the visible school list
4. user navigates to school detail

3.3 School Detail

Main files:

- `src/features/platform/PlatformSchoolDetailPage.tsx`
- `src/features/platform/schoolDetail/hooks/usePlatformSchoolDetailPage.ts`
- `src/features/platform/schoolDetail/components/*`

Flow:

1. route reads `schoolId`
2. dashboard/teachers/students/staff load
3. tab selection is read from query params
4. overview or people tab renders

3.4 Create School Wizard

Main files:

- `src/features/platform/PlatformCreateSchoolPage.tsx`
- `src/features/platform/createSchool/hooks/usePlatformCreateSchool.ts`
- `src/features/platform/createSchool/components/*`
- `src/features/platform/createSchool/helpers/*`

Flow:

1. draft loads from local storage
2. step state is controlled in the hook
3. step validation runs before progress
4. final review step summarizes the payload
5. submit calls school creation
6. successful completion navigates back to schools

Important preserved behavior:

- same draft key
- same step validation rules
- same final payload behavior

4. Management Surface

4.1 Management Dashboard

Main files:

- `src/features/management/ManagementDashboardPage.tsx`
- `src/features/management/dashboard/hooks/useManagementDashboardPage.ts`
- `src/features/management/dashboard/components/*`

Purpose:

- setup-oriented launchpad for management users

Flow:

1. school is resolved from current context or query
2. principal onboarding state is fetched
3. quick links guide setup steps

4.2 Schools Setup

Main files:

- `src/features/management/schools/ManageSchoolsPage.tsx`
- `src/features/management/schools/hooks/useManageSchoolsPage.ts`

Purpose:

- create and view school setup records from management side

Flow:

1. form is submitted
2. create mutation runs
3. success toast is shown
4. list remains visible beneath the form

4.3 Academic Setup: Classes and Sections

Main files:

- `src/features/management/sections/ManageSectionsPage.tsx`
- `src/features/management/sections/hooks/useManageSectionsPage.ts`

Purpose:

- maintain classes and sections

Flow:

1. classes load
2. selected class controls section view
3. create operations invalidate academic setup cache

4.4 Subjects

Main files:

- `src/features/management/setup/SubjectsPage.tsx`
- `src/features/management/setup/subjects/hooks/useSubjectsPage.ts`

Purpose:

- maintain canonical subject catalog

Flow:

1. subjects load from academic setup
2. teaching assignments are fetched to derive usage
3. search filters visible subjects
4. create subject mutation updates the catalog

4.5 Assign Subjects

Main files:

- `src/features/management/setup/AssignSubjectsPage.tsx`
- `src/features/management/setup/assignSubjects/hooks/useAssignSubjectsPage.ts`

Purpose:

- assign teachers to subjects by section

Flow:

1. class and section context is chosen
2. relevant teachers/subjects/assignments load
3. row-level or bulk assignment actions run
4. related assignment caches invalidate

4.6 Principals

Main files:

- `src/features/management/setup/PrincipalsPage.tsx`
- `src/features/management/setup/principals/hooks/usePrincipalsPage.ts`

Purpose:

- principal onboarding and OTP verification management

Flow:

1. management starts onboarding
2. verification modal handles OTP
3. resend and retry actions are available
4. history and current principal state are refreshed

4.7 Teachers Setup

Main files:

- `src/features/management/setup/TeachersPage.tsx`
- `src/features/management/setup/teachers/hooks/useTeachersSetupPage.ts`

Purpose:

- create teacher accounts from management side

Flow:

1. classes and sections load
2. class selection controls available sections
3. submit creates teacher in the active school
4. success state resets the form

4.8 Students Setup and Import

Main files:

- `src/features/management/setup/StudentsPage.tsx`
- `src/features/management/setup/students/hooks/useManagementStudentsPage.ts`
- `src/features/management/setup/students/components/*`

Purpose:

- create students manually
- bulk import students via CSV
- filter and navigate through students

Flow:

1. filters and pagination control the list
2. create modal submits student creation
3. import modal runs preview then commit
4. preview summary, preview table, and final result are separate UI blocks

4.9 Students Setup Read-Only Screen

Main files:

- `src/features/management/setup/StudentsSetupPage.tsx`
- `src/features/management/setup/studentsSetup/hooks/useStudentsSetupPage.ts`

Purpose:

- a lightweight read-only lookup flow using mock sections/students for this phase

5. Principal Surface

5.1 Principal Dashboard

Main files:

- `src/features/principal/PrincipalDashboardPage.tsx`
- `src/features/principal/dashboard/components/*`

Purpose:

- overview of students, teachers, attendance, and notices

5.2 Attendance History

Main files:

- `src/features/principal/attendance/AttendanceHistoryPage.tsx`
- `src/features/principal/attendance/history/hooks/useAttendanceHistoryPage.ts`

Purpose:

- historical attendance overview and breakdowns

Flow:

1. selected date and section determine query
2. summary cards and breakdown table render derived metrics

5.3 Marks History

Main files:

- `src/features/principal/marks/MarksHistoryPage.tsx`
- `src/features/principal/marks/history/hooks/useMarksHistoryPage.ts`

Purpose:

- marks overview, filtered history, export, breakdowns

Flow:

1. filters select section/subject/exam
2. marks query runs
3. summary and tables render

4. export runs on filtered rows only

5.4 Principal Communications

Main files:

- `src/features/principal/PrincipalCommunicationsPage.tsx`
- `src/features/principal/communications/hooks/usePrincipalCommunications.ts`
- `src/features/principal/communications/hooks/usePrincipalCommunicationsNotes.ts`
- `src/features/principal/communications/hooks/usePrincipalCommunicationsDelivery.ts`

Purpose:

- principal-level homework, parent communication, and note entry

Flow:

1. class/section/subject setup loads
2. active tab decides which communication panel renders
3. homework and parent messages use shared tab panels from teacher communications
4. notes flow manages student selection and timeline

6. Teacher Surface

6.1 Teacher Dashboard

Main files:

- `src/features/teachers/dashboard/TeacherDashboard.tsx`
- `src/features/teachers/dashboard/hooks/useTeacherDashboard.ts`

Purpose:

- teacher home screen
- schedule, actions, readiness, navigation handoff

6.2 Mark Attendance

Main files:

- `src/features/teachers/attendance/MarkAttendance.tsx`
- `src/features/teachers/attendance/markAttendance/hooks/useMarkAttendancePag

e.ts`

Purpose:

- teacher attendance recording flow

Flow:

1. attendance section and students load
2. teacher toggles status
3. optimistic state is preserved
4. final submit triggers section/date submission
5. redirect carries toast state

6.3 Enter Marks

Main files:

- `src/features/teachers/marks/EnterMarks.tsx`
- `src/features/teachers/marks/hooks/useEnterMarksPage.ts`

Purpose:

- record exam marks for students

Flow:

1. section/subject/exam context loads
2. existing marks are hydrated
3. max marks and student entries are edited
4. submission runs per-record mark creation then final marks submission
5. route state is sent back to dashboard

6.4 Teacher Communications

Main files:

- `src/features/teachers/TeacherCommunicationsPage.tsx`
- `src/features/teachers/communications/hooks/useTeacherCommunications.ts`
- `src/features/teachers/communications/components/*`

Purpose:

- homework
- parent messaging

- private student notes

Flow:
1. assignment context is established
2. active tab chooses panel
3. composer components collect input
4. timeline components display recent history
5. notes flow lets teacher search student, save note, and review note history

6.5 Teacher List and Teacher Profile

Main files:
- `src/features/teachers/TeachersListPage.tsx`
- `src/features/teachers/TeacherProfilePage.tsx`

Purpose:
- browse teacher roster
- view teacher detail, contact, section, and assignments

7. Student Features

7.1 Students List

Main files:
- `src/features/students/StudentsListPage.tsx`
- `src/features/students/list/hooks/useStudentsListPage.ts`

Purpose:
- role-aware student listing

Teacher mode:
- list students in attendance section

Principal mode:
- filter by section
- search and navigate to profile

7.2 Student Profile

Main files:

- `src/features/students/StudentProfilePage.tsx`
- `src/features/students/profile/hooks/useStudentProfilePage.ts`

Purpose:

- show student personal details
- guardians
- attendance summary
- recent results
- notes
- report card generation

Flow:

1. student profile loads using route id and school context
2. notes load separately
3. report card request runs on demand
4. if backend report fails, a fallback report is generated locally

8. Shared Data Flows

Query Keys

All major shared query identities live in:

- `src/constants/queryKeys.ts`

School Context

All school-scoped APIs should use:

- `requireSchoolId`
- `schoolParams`

Batch Mutations

Attendance and marks use:

- `src/utils/runWithConcurrency.ts`

9. What To Read First for Each Feature

Platform:

- route page
- page hook
- main components

Management:

- route page
- hook
- modal/table components

Principal:

- route page
- hook
- summary components

Teacher:

- route page
- orchestration hook
- panel components

Student:

- profile/list hook
- cards and table/list components

10. Final Summary

If you want to understand the app end to end:

- start at routes
- then layouts
- then one feature hook at a time
- then shared hooks and APIs

The frontend is now organized so each feature can be understood step by step without reading the whole repo at once.